04/03/2026

Categorical Encoding

Handling categorical data comes under Feature transformation of Feature engineering concept.

Generally data is divided into two:

- Data
  - Numerical
  - Categorical
    - Nominal
    - Ordinal.

Numerical data :- The values are in number

Categorical data :- Here the values of the entire column will be like a category like Gender (where you have male, Female, trans). Nationality, Yes/No.

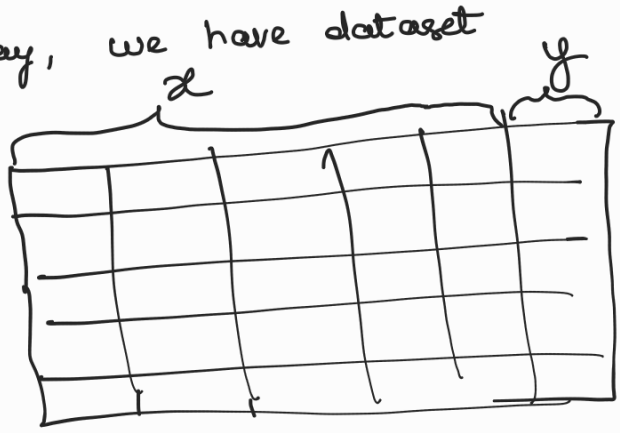
Nominal data :- The values in nominal data is not comparable to one-another and we cannot draw any relationship between them. [eg:- state, country, Gender)

Ordinal data :- The values in ordinal data have some relationship b/w them, where we can categorize as higher/lower (eg:- Marks - Merit/distinction) Pass/Fail)

The categorical dataset are in strings, but machine learning understand only numerical dataset. Hence we need to convert categories into numerical value by a method called **Encoding**.

* Ordinal data is converted using ordinal encoding
* Nominal data is converted using one hot encoding
* Label encoding is also similar to ordinal encoding

Let's say, we have dataset



$x \rightarrow$ Independent feature (Input)

$y \rightarrow$ Dependent feature (output)

* If there is any categorical data in x ie., input we use ordinal encoding
* If there is any categorical data in y ie., output we use Label encoding

Example :

Age	Gender	Review	Education	output
20	Female	Good	PG	Yes
30	Male	Bad	PG	Yes
50	Female	Average	UG	No
65	Female	Average	PG	No
72	Male	Good	School	No
⋮	⋮	⋮	⋮	⋮

x y

In x, Gender → Nominal data (One hot encoding)

Review → ordinal data
Education → ordinal data } ordinal encoding

In y, output → Nominal data [since it is output y we use Label encoding]

Code :- Ordinal Encoder

⇒ from sklearn.Preprocessing import OrdinalEncoder

orders are given in list

⇒ oe = OrdinalEncoder (categories = [['Poor', 'Average', 'Good'], ['School', 'UG', 'PG']])

oe.fit (X_train)
X_train = oe.transform (X_train)
X_test = oe.transform (X_test)

fitting only in training

transform done both in train & test.

Label encoder

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

le.fit (y_train)

y_train = le.transform (y_train)
y_test = le.transform (y_test)

orders not required to be given. It takes on its own.